

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Ishita Roy (1WA24CS131)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Ishita Roy (1WA24CS131)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-5
2	Implement Johnson Trotter algorithm to generate permutations.	6-7
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	8-10
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	11-15
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	14-16
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	17-18
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	19-20
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	21-24
9	Implement Fractional Knapsack using Greedy technique.	25-27
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	28-29
11	Implement "N-Queens Problem" using Backtracking.	30-31

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int adj[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int n;

void topologicalSort()
{
    int front = 0, rear = -1;
    int count = 0;

    /* Initialize indegree array */
    for (int i = 0; i < n; i++)
    {
        indegree[i] = 0;
    }

    /* Calculate indegree of each vertex */
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (adj[i][j] == 1)
            {
                indegree[j]++;
            }
        }
    }

    /* Insert all vertices with indegree 0 into queue */
    for (int i = 0; i < n; i++)
    {
        if (indegree[i] == 0)
        {
            queue[++rear] = i;
        }
    }
}
```

```

    }

    printf("\nTopological Order: ");

    while (front <= rear)
    {
        int node = queue[front++];

        printf("%d ", node);
        count++;

        for (int i = 0; i < n; i++)
        {
            if (adj[node][i] == 1)
            {
                indegree[i]--;

                if (indegree[i] == 0)
                {
                    queue[++rear] = i;
                }
            }
        }
    }

    if (count != n)
    {
        printf("\nGraph has a cycle! No topological ordering possible.\n");
    }
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &adj[i][j]);
        }
    }
}

```

```
topologicalSort();
```

```
return 0;
```

```
}
```

OUTPUT

```
Enter number of vertices: 4

Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological Order: 0 1 2 3
PS C:\Users\student\Documents\1\output>
```

Leetcode 207: Course Schedule (Topological Sort)

The screenshot displays the LeetCode interface for problem 207, "Course Schedule". The problem description states that there are `numCourses` courses, each with a list of prerequisites. The goal is to determine if it's possible to finish all courses. The solution provided is a C++ program using a topological sort algorithm (BFS). The code initializes an adjacency list and an indegree array, then processes nodes with zero indegree until all courses are completed. The test results on the right show the solution is "Accepted" with a runtime of 66 ms and memory usage of 126.24 MB.

207. Course Schedule Solved

Medium Topics Companies Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:
Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `true`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.

Example 2:
Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output: `false`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- All the pairs `prerequisites[i]` are unique.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
6
7     // Create adjacency matrix
8     int** graph = (int**)malloc(numCourses * sizeof(int*));
9     for(int i = 0; i < numCourses; i++) {
10         graph[i] = (int*)calloc(numCourses, sizeof(int));
11     }
12
13     // Indegree array
14     int* indegree = (int*)calloc(numCourses, sizeof(int));
15
16     // Build graph
17     for(int i = 0; i < prerequisitesSize; i++) {
18         int a = prerequisites[i][0];
19         int b = prerequisites[i][1];
20
21         if(graph[b][a] == 0) {
22             graph[b][a] = 1;
23             indegree[a]++;
24         }
25     }
26
27     // Queue
28     int* queue = (int*)malloc(numCourses * sizeof(int));
29     int front = 0, rear = 0;
30
31     // Push all nodes with indegree 0
32     for(int i = 0; i < numCourses; i++) {
33         if(indegree[i] == 0) {
34             queue[rear++] = i;
35         }
36     }
37
38     int count = 0;
39
40     // BFS
41     while(front < rear) {
42         int node = queue[front++];
```

Test Result Testcase

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`numCourses = 2`

`prerequisites = [[1,0]]`

Output

`true`

Expected

Submissions Accepted

All Submissions

Accepted 84 / 84 testcases Passed

Submitted at May 21, 2026 19:10

Analysis Solution

Runtime 66 ms Beats 5.39%

Memory 126.24 MB Beats 5.68%

100% 0% 5% 10% 15%

0% 20% 40% 60% 80% 100%

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

SOURCE CODE

```
#include <stdio.h>
int a[10], dir[10];

void print(int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int mobile(int n)
{
    int m = 0, pos = -1;
    for (int i = 0; i < n; i++)
    {
        int j = i + dir[i];
        if (j >= 0 && j < n &&
            a[i] > a[j] &&
            a[i] > m)
        {
            m = a[i];
            pos = i;
        }
    }
    return pos;
}

int main()
{
    int n;
    printf("Enter number: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i] = -1; // Initially all directions point left
    }
    print(n);
```

```

while (1)
{
    int i = mobile(n);

    if (i == -1)
        break;

    int j = i + dir[i];

    // Swap elements
    int temp = a[i];
    a[i] = a[j];
    a[j] = temp;

    // Swap directions
    temp = dir[i];
    dir[i] = dir[j];
    dir[j] = temp;

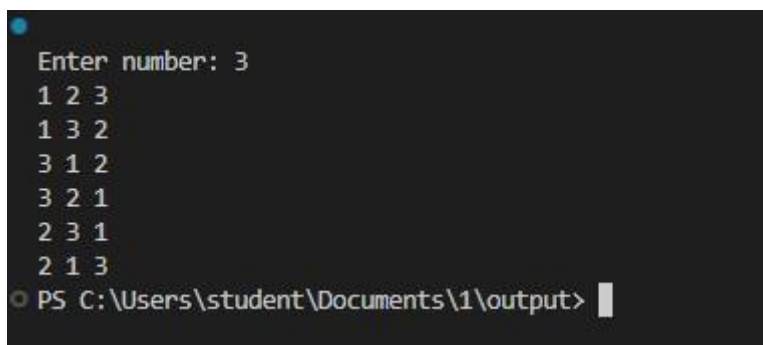
    // Reverse directions of all elements larger than moved element
    for (int k = 0; k < n; k++)
    {
        if (a[k] > a[j])
            dir[k] *= -1;
    }

    print(n);
}

return 0;
}

```

OUTPUT



```

Enter number: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
PS C:\Users\student\Documents\1\output>

```


Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[5000000], b[5000000];
void merge(int a[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = low;
    while (i <= mid && j <= high)
    {
        if (a[i] < a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }
    while (i <= mid)
    {
        b[k++] = a[i++];
    }
    while (j <= high)
    {
        b[k++] = a[j++];
    }
    for (int x = low; x <= high; x++)
    {
        a[x] = b[x];
    }
}

void merge_sort(int a[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;
        merge_sort(a, low, mid);
        merge_sort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}
```

```

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("\nOriginal Array:\n");
    for (int i = 0; i < n; i++)
    {
        a[i] = rand() % 1000;
        printf("%d ", a[i]);
    }
    clock_t start_time = clock();
    merge_sort(a, 0, n - 1);
    clock_t end_time = clock();
    printf("\n\nSorted Array:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    float total_time = (float)(end_time - start_time) * 1000 / CLOCKS_PER_SEC;
    printf("\n\nTime Taken: %.2f ms\n", total_time);

    return 0;
}

```

OUTPUT

```

Enter number of elements: 10

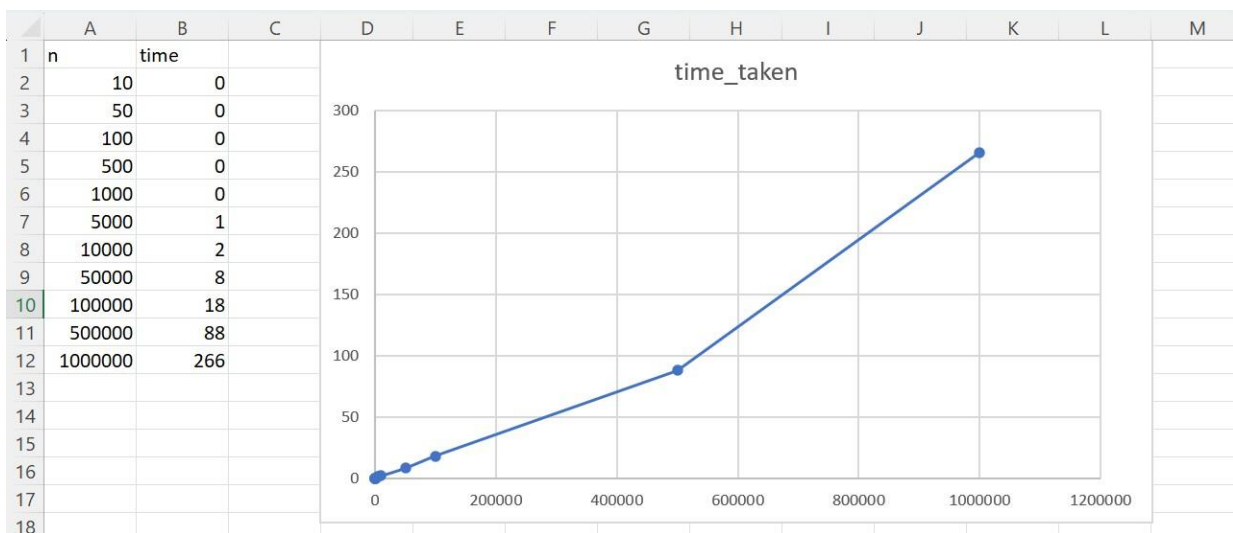
Entering the elements...41 467 334 500 169 724 478 358 962 464
After sorting: 41 169 334 358 464 467 478 500 724 962
Time taken is: 0.00
PS C:\Users\student\Documents\1\output>

```

Time taken for each N

N	Time taken
10	0.0
50	0.0
100	0.0
500	0.0
1000	0.0
5000	1.0
10000	2.0
50000	8.0
100000	18.0
500000	88.0
1000000	266.0

Graph:



Leetcode 148: Sort List

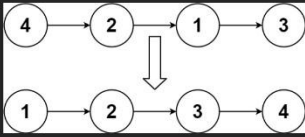
148. Sort List

Solved

Medium Topics Companies

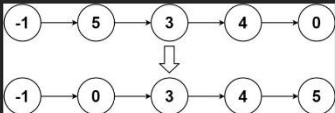
Given the head of a linked list, return the list after sorting it in ascending order.

Example 1:



Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Example 3:

Input: head = []
Output: []

13.1K 186 96 Online

Code

```
1  /*
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     struct ListNode *next;
6  * };
7  */
8  #include <stdio.h>
9  #include <stdlib.h>
10
11 // Definition for singly-linked list.
12 struct ListNode {
13     int val;
14     struct ListNode *next;
15 };
16
17 // Merge two sorted lists
18 struct ListNode* merge(struct ListNode* l1, struct ListNode* l2) {
19
20     struct ListNode dummy;
21     struct ListNode* tail = &dummy;
22
23     dummy.next = NULL;
24
25     while(l1 && l2) {
26
27         if(l1->val < l2->val) {
28             tail->next = l1;
29             l1 = l1->next;
30         } else {
31             tail->next = l2;
32             l2 = l2->next;
33         }
34         tail = tail->next;
35     }
36
37     if(l1)
38         tail->next = l1;
39     else
40         tail->next = l2;
41 }
```

Ln 1, Col 1

Test Result

Testcase

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

head =

[4,2,1,3]

Output

[1,2,3,4]

Expected

[1,2,3,4]

Submissions Accepted

All Submissions

Accepted 30/30 testcases passed

Analysis Solution

submitted at May 21, 2026 20:

Runtime

13 ms Beats 67.81%

Memory

25.32 MB Beats 87.61%

0% 10% 15% 20%

0ms 11ms 16ms 21ms

Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int a[5000000];
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int a[], int low, int high)
{
    int pivot = a[low];
    int i = low + 1;
    int j = high;
    while (i <= j)
    {
        while (i <= high && pivot >= a[i])
            i++;
        while (j >= low && pivot < a[j])
            j--;
        if (i < j)
```

```

        swap(&a[i], &a[j]);
    }
    swap(&a[low], &a[j]);
    return j;
}

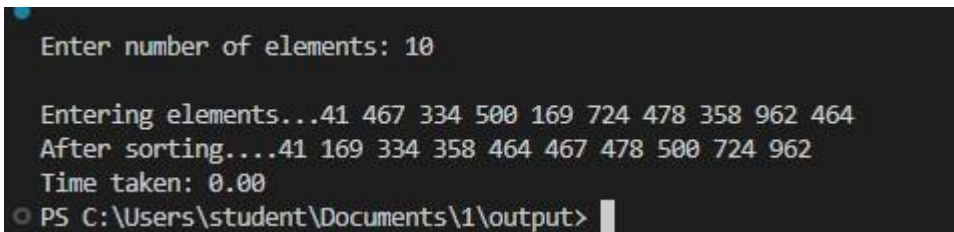
void quick_sort(int a[], int low, int high)
{
    if (low < high)
    {
        int j = partition(a, low, high);
        quick_sort(a, low, j - 1);
        quick_sort(a, j + 1, high);    }
    }

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("\nOriginal Array:\n");
    srand(time(NULL));
    for (int i = 0; i < n; i++)
    {
        a[i] = rand() % 1000;
        printf("%d ", a[i]);
    }
    clock_t start = clock();
    quick_sort(a, 0, n - 1);
    clock_t end = clock();
    printf("\n\nSorted Array:\n");
    for (int i = 0; i < n; i++)

```

```
{  
    printf("%d ", a[i]);  
}  
  
float time_taken =  
    ((float)(end - start) / CLOCKS_PER_SEC) * 1000;  
  
printf("\n\nTime Taken: %.2f ms\n", time_taken);  
return 0;  
}
```

OUTPUT



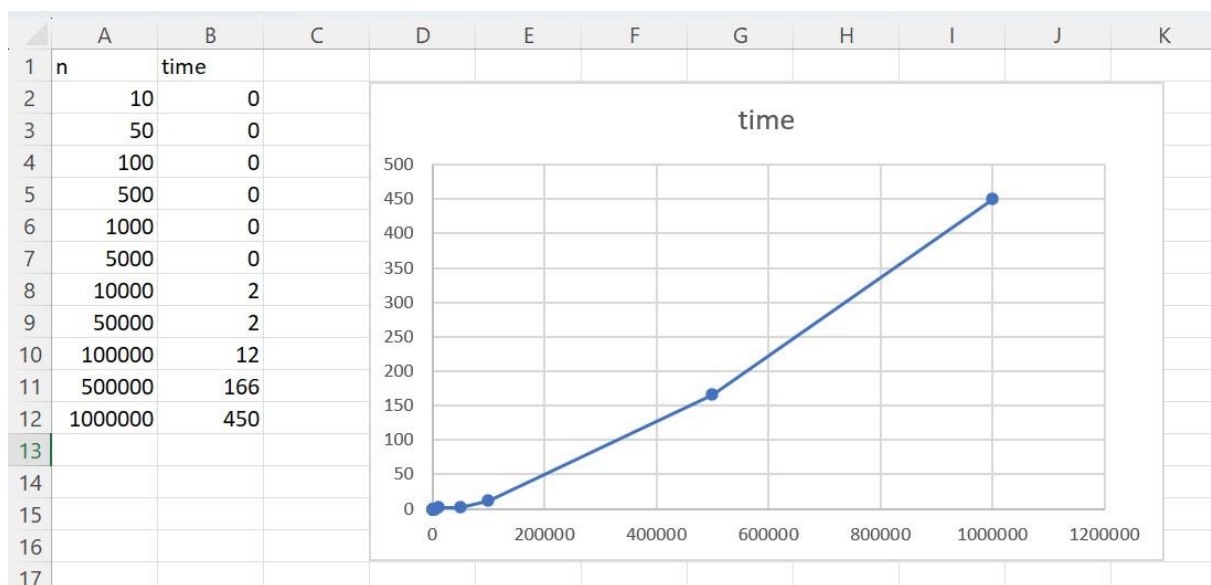
A screenshot of a terminal window with a dark background. The text is as follows:
Enter number of elements: 10

Entering elements...41 467 334 500 169 724 478 358 962 464
After sorting....41 169 334 358 464 467 478 500 724 962
Time taken: 0.00
PS C:\Users\student\Documents\1\output>

Time taken for each N

N	Time taken
10	0.0
50	0.0
100	0.0
500	0.0
1000	0.0
5000	0.0
10000	2.0
50000	2.0
100000	12.0
500000	166.0
1000000	450.0

Graph:



Leetcode 215: kth Largest Element in an Array

215. Kth Largest Element in an Array Solved

Medium Topics Companies

Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:
Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: `5`

Example 2:
Input: `nums = [3,2,3,1,2,4,5,6]`, `k = 4`
Output: `4`

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Seen this question in a real interview before? 1/6

Yes No

Accepted **3,657,729** / 3.8M Acceptance Rate **69.0%**

Topics Companies

18.9K 393 179 Online

Restored from local. Upgrade to Cloud Saving

Le 11, Col 32

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Comparator for descending order
5 int compare(const void *a, const void *b) {
6     return (*(int*)b - *(int*)a);
7 }
8
9 int findKthLargest(int* nums, int numsSize, int k) {
10
11     // Sort in descending order
12     qsort(nums, numsSize, sizeof(int), compare);
13
14     // kth largest element
15     return nums[k - 1];
16 }
```

Test Result Runtime: 0 ms

Accepted

Case 1 Case 2

Input

nums =
[3,2,1,5,6,4]

k =
2

Output

5

Expected

5

Accepted

All Submissions

Accepted 45 / 45 testcases passed

Jahna_Boy_ submitted at May 21, 2026 20:03

Runtime

54 ms Beats 13.19%

Submissions

Status	Language	Runtime	Memory
Accepted	C	54 ms	16.8 M

Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[5000000];
void heapify(int a[], int i, int n)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i)
    {
        int temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;
        heapify(a, largest, n);
    }
}
void heap_sort(int a[], int n)
{
    /* Build Max Heap */
    for (int i = (n / 2) - 1; i >= 0; i--)
    {
        heapify(a, i, n);
    }
    /* Extract elements one by one */
    for (int i = n - 1; i > 0; i--)
    {
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, 0, i);
    }
}
```

```

void print(int a[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
}

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    srand(time(NULL));
    printf("\nOriginal Array:\n");
    for (int i = 0; i < n; i++)
    {
        a[i] = rand() % 1000;
    }
    print(a, n);
    clock_t start = clock();
    heap_sort(a, n);
    clock_t end = clock();
    printf("\nSorted Array:\n");
    print(a, n);
    float time_taken =
        ((float)(end - start) / CLOCKS_PER_SEC) * 1000;
    printf("\nTime Taken: %.2f ms\n", time_taken);

    return 0;
}

```

OUTPUT

```
Enter number of elements: 5

Entering elements...41 467 334 500 169

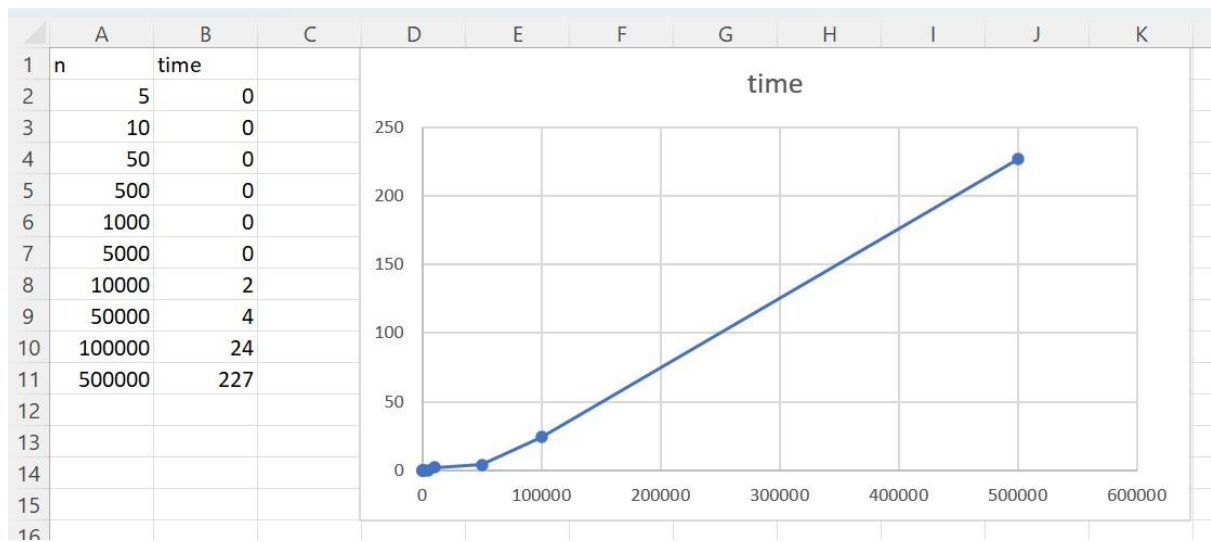
After sorting....41 169 334 467 500

Time taken: 0.00
PS C:\Users\student\Documents\1\output> |
```

Time taken for each N

N	Time taken
5	0
10	0
50	0
500	0
1000	0
5000	0
10000	2
50000	4
100000	24
500000	227
5	0

Graph:



Lab Program 6: Implement 0/1 Knapsack problem using dynamic programming

SOURCE CODE

```
#include <stdio.h>
int p[50], w[50], n, m;

void inputs()
{
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter knapsack capacity: ");
    scanf("%d", &m);
    printf("Enter weights of items:\n");
    for (int i = 1; i <= n; i++)
    {
        scanf("%d", &w[i]);
    }
    printf("Enter profits of items:\n");
    for (int i = 1; i <= n; i++)
    {
        scanf("%d", &p[i]);
    }
}

int max(int a, int b)
{
    return (a > b) ? a : b;
}

int knapsack()
{
    int v[n + 1][m + 1];

    for (int i = 0; i <= n; i++)
    {
        for (int j = 0; j <= m; j++)
        {
            if (i == 0 || j == 0)
            {
                v[i][j] = 0;
            }
            else if (w[i] > j)
            {
                v[i][j] = v[i - 1][j];
            }
            else
            {

```

```

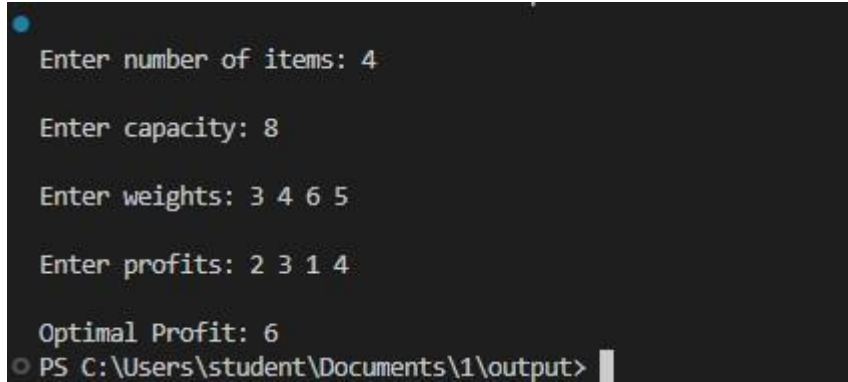
        v[i][j] = max(
            v[i - 1][j],
            v[i - 1][j - w[i]] + p[i]
        );
    }
}

return v[n][m];
}

int main()
{
    inputs();
    int result = knapsack();
    printf("\nOptimal Profit = %d\n", result);
    return 0;
}

```

OUTPUT



```

Enter number of items: 4

Enter capacity: 8

Enter weights: 3 4 6 5

Enter profits: 2 3 1 4

Optimal Profit: 6
PS C:\Users\student\Documents\1\output>

```

Leetcode 416 : Partition Equal Subset Sum

416. Partition Equal Subset Sum Solved

Medium Topics Companies

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `false` otherwise.

Example 1:
Input: `nums = [1,5,11,5]`
Output: `true`
Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:
Input: `nums = [1,2,3,5]`
Output: `false`
Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 100`

Seen this question in a real interview before? 1/6

Yes No

Accepted 1,528,524 / 3.1M Acceptance Rate 49.5%

Topics Companies

13.9K 255 120 Online

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 bool canPartition(int* nums, int numsSize) {
5
6     int sum = 0;
7
8     // Find total sum
9     for(int i = 0; i < numsSize; i++) {
10         sum += nums[i];
11     }
12
13     // If sum is odd, partition not possible
14     if(sum % 2 != 0)
15         return false;
16
17     int target = sum / 2;
18
19     // DP array
20     bool dp[target + 1];
21
22     for(int i = 0; i <= target; i++) {
23         dp[i] = false;
24     }
25
26     dp[0] = true;
27
28     // Subset sum DP
29     for(int i = 0; i < numsSize; i++) {
30
31         for(int j = target; j >= nums[i]; j--) {
32
33             if(dp[j - nums[i]]) {
34                 dp[j] = true;
35             }
36         }
37     }
38
39     return dp[target];
40 }
```

Test Result Runtime: 0 ms

Accepted Case 1 Case 2

Input

`nums =`

`[1,5,11,5]`

Output

`true`

Expected

`true`

Contribute a testcase

Submissions Accepted x

All Submissions

Accepted 147 / 147 testcases passed

submitted at May 21, 2026 20:1

Analysis Solution

Runtime 37 ms | Beats 67.77%

Memory 8.86 MB | Beats 47.93%

15% 10% 5% 0%

10ms 20ms 30ms 40ms 50ms 60ms 70ms

Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

SOURCE CODE

```
#include <stdio.h>

#define INF 99999

void floyd(int n, int R[n][n])
{
    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (R[i][j] > R[i][k] + R[k][j])
                {
                    R[i][j] = R[i][k] + R[k][j];
                }
            }
        }
    }
}

int main()
{
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int R[n][n];
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &R[i][j]);
        }
    }
}
```

```

    }
}

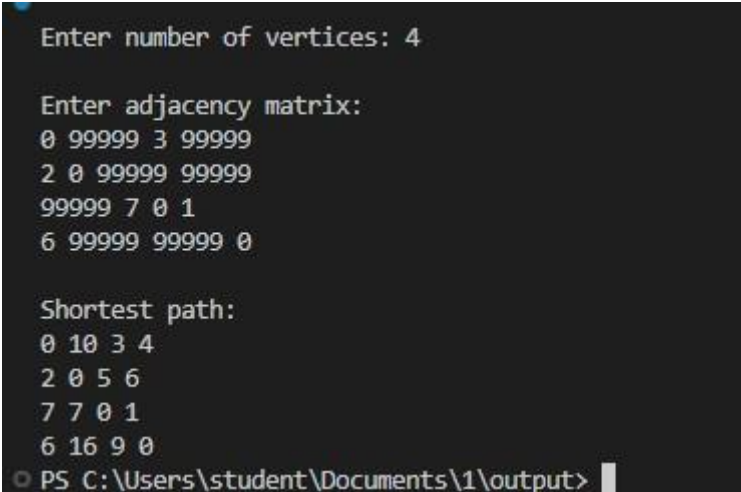
floyd(n, R);
printf("\nAll-Pairs Shortest Path Matrix:\n");

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (R[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", R[i][j]);
    }
    printf("\n");
}

return 0;
}

```

OUTPUT



```

Enter number of vertices: 4

Enter adjacency matrix:
0 99999 3 99999
2 0 99999 99999
99999 7 0 1
6 99999 99999 0

Shortest path:
0 10 3 4
2 0 5 6
7 7 0 1
6 16 9 0
PS C:\Users\student\Documents\1\output>

```

Leetcode 1334 : Find the city with smallest number of neighbours at a threshold distance

Problem List < > > >

Description Editorial Solutions

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

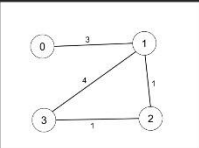
Medium Topics Companies Hint

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is **at most** `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities `i` and `j` is equal to the sum of the edges' weights along that path.

Example 1:



Input: `n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`, `distanceThreshold = 4`
Output: 3
Explanation: The figure above describes the graph. The neighboring cities at a `distanceThreshold = 4` for each city are:
City 0 $\rightarrow$ [City 1, City 2]
City 1 $\rightarrow$ [City 0, City 2, City 3]
City 2 $\rightarrow$ [City 0, City 1, City 3]
City 3 $\rightarrow$ [City 0, City 1, City 2]

3.6K 229 40 Online Saved

Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define INF 100000000
5
6 int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {
7
8     int dist[n][n];
9
10    // Initialize distances
11    for(int i = 0; i < n; i++) {
12        for(int j = 0; j < n; j++) {
13
14            if(i == j)
15                dist[i][j] = 0;
16            else
17                dist[i][j] = INF;
18        }
19    }
20
21    // Fill edges
22    for(int i = 0; i < edgesSize; i++) {
23
24        int u = edges[i][0];
25        int v = edges[i][1];
26        int w = edges[i][2];
27
28        dist[u][v] = w;
29        dist[v][u] = w;
30    }
31
32    // Floyd Marshall
33    for(int k = 0; k < n; k++) {
34        for(int i = 0; i < n; i++) {
35            for(int j = 0; j < n; j++) {
36
37                if(dist[i][k] + dist[k][j] < dist[i][j]) {
38                    dist[i][j] = dist[i][k] + dist[k][j];
39                }
40            }
41        }
42    }
43 }
```

Ln 1, Col 1

Test Result Testcase

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`n =`
4

`edges =`
[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

`distanceThreshold =`
4

Output

Submissions Accepted x

All Submissions

Accepted 54 / 54 testcases passed

submitted at May 21, 2026 20:11

Analysis Solution

Runtime

11 ms Beats 81.35%

Memory

11.38 MB Beats 67.71%



Lab Program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

SOURCE CODE

```
#include <stdio.h>

void prims(int n, int graph[n][n])
{
    int visited[n];

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    int edges = 0;
    int cost = 0;

    visited[0] = 1;

    printf("\nEdge\tWeight\n");

    while (edges < n - 1)
    {
        int min = 100;
        int x = -1, y = -1;

        for (int i = 0; i < n; i++)
        {
            if (visited[i] == 1)
            {
                for (int j = 0; j < n; j++)
                {
                    if (visited[j] == 0 && graph[i][j] != 0)
                    {
                        if (graph[i][j] < min)
                        {
                            min = graph[i][j];
                            x = i;
                            y = j;
                        }
                    }
                }
            }
        }
    }
}
```

```

    }

    printf("%d - %d\t%d\n", x, y, graph[x][y]);

    cost += graph[x][y];
    visited[y] = 1;
    edges++;
}

printf("\nMinimum Cost = %d\n", cost);
}

int main()
{
    int n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int graph[n][n];

    printf("Enter the adjacency matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    prims(n, graph);

    return 0;
}

```

OUTPUT

```
Enter number of vertices: 6
```

```
Enter the adjacency matrix:
```

```
0 9 8 7 6 5  
4 3 2 4 3 1  
2 4 3 1 4 2  
1 2 3 4 5 6  
2 5 3 6 5 4  
2 4 5 3 2 1
```

```
Edge    Weight
```

```
0 - 5    5  
5 - 4    2  
4 - 2    3  
2 - 3    1  
3 - 1    2
```

```
Minimum Cost 13
```

```
PS C:\Users\student\Documents\1\output>
```

Lab Program 8:

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

SOURCE CODE

```
#include <stdio.h>

struct Edge{
int u,v,w;
}e[10],temp;

int parent[10];

int find(int x){
while(parent[x]!=x)
x=parent[x];    return
x;
}

int main(){

    int n,m,cost=0;

    printf("\nEnter number of Vertices and Edges: ");
    scanf("%d%d",&n,&m);

    printf("Enter vertex u v and weight w:\n");

    for(int i=0;i<m;i++)
scanf("%d%d%d",&e[i].u,&e[i].v,&e[i].w);

    for(int i=0;i<m-1;i++)
for(int j=0;j<m-i-1;j++)
if(e[j].w > e[j+1].w){
    temp=e[j];
e[j]=e[j+1];
e[j+1]=temp;
}

    for(int i=0;i<n;i++)
```

```

    parent[i]=i;

    printf("\nMST:\n");

    for(int i=0,edges=0;i<m && edges<n-1;i++){

        int u=find(e[i].u);
        int v=find(e[i].v);
        if(u!=v){
            printf("%d %d\n",e[i].u,e[i].v,e[i].w)
            ;

            cost+=e[i].w;
            edges++;

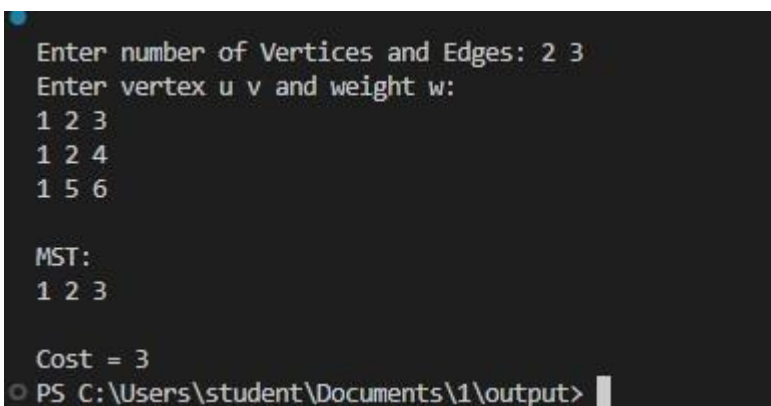
            parent[u]=v;
        }
    }

    printf("\nCost = %d",cost);

    return 0;
}

```

OUTPUT



```

Enter number of Vertices and Edges: 2 3
Enter vertex u v and weight w:
1 2 3
1 2 4
1 5 6

MST:
1 2 3

Cost = 3
PS C:\Users\student\Documents\1\output>

```


Lab program 9:

Implement Fractional Knapsack using Greedy technique.

SOURCE CODE

```
#include <stdio.h>

int main()
{
    int n, rem_weight;
    float total_profit = 0;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int obj[n], p[n], w[n];
    printf("Enter objects: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &obj[i]);
    printf("Enter profits: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &p[i]);
    printf("Enter weights: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &w[i]);
    printf("Enter knapsack capacity: ");
    scanf("%d", &rem_weight);
    float pw[n];
    /* Calculate Profit/Weight Ratio */
    for (int i = 0; i < n; i++)
    {
        pw[i] = (float)p[i] / w[i];
    }
    /* Sort in descending order of Profit/Weight Ratio */
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (pw[i] < pw[j])
            {
                float temp_pw = pw[i];
                pw[i] = pw[j];
                pw[j] = temp_pw;

                int temp = obj[i];
                obj[i] = obj[j];
                obj[j] = temp;
            }
        }
    }
}
```

```

        temp = p[i];
        p[i] = p[j];
        p[j] = temp;

        temp = w[i];
        w[i] = w[j];
        w[j] = temp;
    }
}
}

/* Fractional Knapsack Calculation */
for (int k = 0; k < n; k++)
{
    if (rem_weight >= w[k])
    {
        rem_weight -= w[k];
        total_profit += p[k];
    }
    else
    {
        total_profit += ((float)rem_weight / w[k]) * p[k];
        break;
    }
}
printf("\nFractional Knapsack:\n");
printf("Object\tProfit\tWeight\tP/W Ratio\n");
for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%.2f\n",
        obj[i], p[i], w[i], pw[i]);
}
printf("\nTotal Profit = %.2f\n", total_profit);

return 0;
}

```

OUTPUT

```
Enter number of item: 5

Enter objects: 1
2
3
4
5

Enter profits: 25 20 15 40 50

Enter weights: 3 2 1 4 5

Enter remaining weight: 6

Fractional Knapsack:
Object  Profit  Weight
3       15     1

2       20     2

4       40     4

5       50     5

1       25     3

Total Profit is: 65.000000
PS C:\Users\student\Documents\1\output> 
```

Leetcode 55: Jump Game

Problem List

55. Jump Game

Solved

Medium

Topics

Companies

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

Input: `nums = [2,3,1,1,4]`
Output: `true`
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`
Output: `false`
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $0 \leq \text{nums}[i] \leq 10^5$

Discover more [Programming logic puzzles](#)

Seen this question in a real interview before? 1/6

Yes No

Accepted 3,240,202 / 3M Acceptance Rate 41.0%

219K 511 172 Online

Code

```
1 #include <stdbool.h>
2
3 bool canJump(int* nums, int numsSize)
4 {
5     int maxReach = 0;
6     for (int i = 0; i < numsSize; i++)
7     {
8         if (i > maxReach)
9             return false;
10
11         if (i + nums[i] > maxReach)
12             maxReach = i + nums[i];
13     }
14     return true;
15 }
16
17 }
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums = [2,3,1,1,4]

Output

true

Expected

true

Contribute a testcase

Submissions Accepted

All Submissions

Accepted 176 / 176 testcases passed

Jahira_Roy, submitted at Jun 05, 2028 23:03

Runtime

0 ms Beats 100.00%

Memory

11.28 MB Beats 69.43%

60%

40%

20%

0%

10.8mb 11mb 11.1mb 11.2mb 11.3mb

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

SOURCE CODE

```
#include <stdio.h>

#define INF 99999

int main()
{
    int n, src;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int cost[n][n], dist[n], vis[n];
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &src);
    /* Initialize distance and visited arrays */
    for (int i = 0; i < n; i++)
    {
        dist[i] = cost[src][i];
        vis[i] = 0;
    }

    dist[src] = 0;
    vis[src] = 1;

    for (int i = 1; i < n; i++)
    {
        int min = INF;
        int u = -1;

        /* Find the unvisited vertex with minimum distance */
        for (int j = 0; j < n; j++)
        {
            if (!vis[j] && dist[j] < min)
            {
```

```

        min = dist[j];
        u = j;
    }
}

if (u == -1)
    break; /* No reachable vertex left */

vis[u] = 1;

/* Update distances of adjacent vertices */
for (int v = 0; v < n; v++)
{
    if (!vis[v] &&
        cost[u][v] != INF &&
        dist[u] + cost[u][v] < dist[v])
    {
        dist[v] = dist[u] + cost[u][v];
    }
}
}

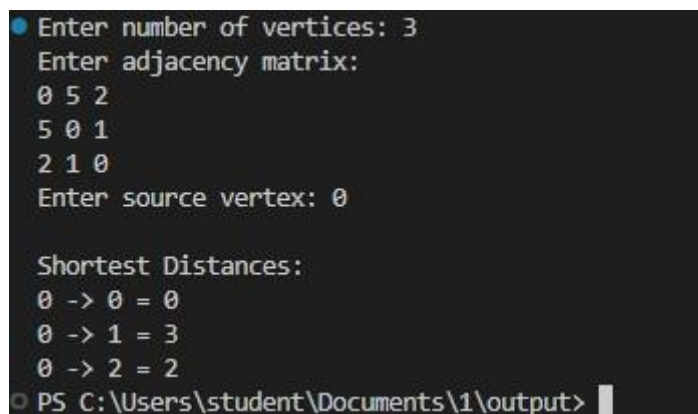
printf("\nShortest Distances from Vertex %d:\n", src);

for (int i = 0; i < n; i++)
{
    printf("%d -> %d = %d\n", src, i, dist[i]);
}

return 0;
}

```

OUTPUT



```

Enter number of vertices: 3
Enter adjacency matrix:
0 5 2
5 0 1
2 1 0
Enter source vertex: 0

Shortest Distances:
0 -> 0 = 0
0 -> 1 = 3
0 -> 2 = 2
PS C:\Users\student\Documents\1\output>

```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>

int x[20], n;

int place(int k, int i)
{
    for (int j = 1; j < k; j++)
    {
        if (x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }

    return 1;
}

void nqueen(int k)
{
    for (int i = 1; i <= n; i++)
    {
        if (place(k, i))
        {
            x[k] = i;

            if (k == n)
            {
                printf("\nSolution:\n");

                for (int r = 1; r <= n; r++)
                {
                    for (int c = 1; c <= n; c++)
                    {
                        if (x[r] == c)
                            printf("Q ");
                        else
                            printf(". ");
                    }
                    printf("\n");
                }

                printf("\n");
            }
            else

```

```

        {
            nqueen(k + 1);
        }
    }
}

int main()
{
    printf("Enter value of n: ");
    scanf("%d", &n);

    nqueen(1);

    return 0;
}

```

OUTPUT

```

● Enter value of n: 4

Solution:
. Q . .
. . . Q
Q . . .
. . Q .

Solution:
. . Q .
Q . . .
. . . Q
. Q . .

PS C:\Users\student\Documents\1\output>

```